

Chapitre 5 : Structures de données de base

Oscar Junior WAGOUSSI

ENEAM

March 3, 2025

oscarjrw@gmail.com

Objectifs du Chapitre 5

- ▶ Maîtriser la création et la manipulation des listes en Python.
- ▶ Effectuer des opérations sur les chaînes de caractères (concaténation, extraction de sous-chaînes).
- ▶ Utiliser les dictionnaires pour stocker et manipuler des données sous forme de paires clé-valeur.
- ▶ Comprendre les propriétés des tuples et des ensembles et savoir les utiliser dans vos programmes.

Les listes en Python

- ▶ Une liste est une collection ordonnée d'éléments.
- ▶ Une liste (ou list / array) en python est une variable dans laquelle on peut mettre plusieurs variables.
- ▶ On peut créer, modifier, ajouter ou supprimer des éléments.
- ▶ On peut ajouter les valeurs souhaitées lors de la création de la liste.
- ▶ Il est possible d'accéder aux éléments d'une liste en utilisant leur index.
- ▶ Les listes sont itérables, ce qui permet de parcourir leurs éléments avec une boucle for.

Créer et accéder à une liste

Listing 1: Création et accès aux éléments d'une liste

```
1 # Création d'une liste
2 liste = [1, 2, 3]
3 print(liste) # [1, 2, 3]
4
5 # Accéder à un élément
6 liste = ["a", "d", "m"]
7 print(liste[0]) # 'a'
8 print(liste[2]) # 'm'
9
```

Méthodes des listes

Voici quelques méthodes importantes des listes en Python :

- ▶ `list.append(x)` : ajoute un élément à la fin de la liste.
- ▶ `list.extend(iterable)` : étend la liste avec un autre iterable.
- ▶ `list.insert(i, x)` : insère un élément à l'index spécifié.
- ▶ `list.remove(x)` : supprime la première occurrence de l'élément.
- ▶ `list.pop([i])` : enlève l'élément à l'index donné (dernier élément par défaut).
- ▶ `list.clear()` : vide la liste.
- ▶ `list.index(x, [start, end])` : renvoie l'index de l'élément.
- ▶ `list.count(x)` : compte les occurrences d'un élément.
- ▶ `list.sort(*, key=None, reverse=False)` : trie la liste.
- ▶ `list.reverse()` : inverse l'ordre des éléments.
- ▶ `list.copy()` : crée une copie de la liste.

list.append(x)

Ajoute un élément à la fin de la liste. Équivalent à `a[len(a):] = [x]`.

Listing 2: list.append(x)

```
1 >>> liste = []
2 >>> liste
3 []
4 >>> liste.append(1)
5 >>> liste
6 [1]
7 >>> liste.append("ok")
8 >>> liste
9 [1, 'ok']
10
```

`list.extend(iterable)`

Étend la liste en y ajoutant tous les éléments de l'itérable.
Équivalent à `a[len(a):] = iterable`.

Listing 3: `list.extend(iterable)`

```
1 >>> x = [1, 2, 3, 4]
2 >>> y = [4, 5, 1, 0]
3 >>> x.extend(y)
4 >>> print(x)
5 [1, 2, 3, 4, 4, 5, 1, 0]
6
```

```
list.insert(i, x)
```

Insère un élément à la position indiquée.

Listing 4: Insère un élément à la position indiquée

```
1 # Création d'une liste de voyelles
2 vowel = ['a', 'e', 'i', 'u']
3
4 # 'o' est inséré à l'index 3 (4ème position)
5 vowel.insert(3, 'o')
6
```


`list.remove(x)`

Supprime de la liste le premier élément dont la valeur est égale à `x`.

Listing 5: `list.remove(x)`

```
1 >>> liste = ["a", "b", "c"]
2 >>> liste.remove("a")
3 >>> liste
4 ['b', 'c']
5
```

list.pop([i])

Enlève de la liste l'élément situé à la position indiquée et le renvoie en valeur de retour.

Listing 6: list.pop([i

```
1 # Création d'une liste de nombres premiers
2 prime_numbers = [2, 3, 5, 7]
3
4 # Suppression de l'élément à l'index 2
5 removed_element = prime_numbers.pop(2)
6
```

list.clear()

Supprime tous les éléments de la liste. Équivalent à `del a[:]`.

Listing 7: list.clear()

```
1 # Création d'une liste de nombres premiers
2 prime_numbers = [2, 3, 5, 7, 9, 11]
3
4 # Suppression de tous les éléments
5 prime_numbers.clear()
6
```

```
list.index(x, [start, end])
```

Renvoie la position du premier élément de la liste dont la valeur est égale à x.

Listing 8: Renvoie la position du premier élément correspondant

```
1 >>> liste = ["a", "a", "a", "b", "c", "c"]
2 >>> liste.index("b")
3 3
4
```

list.count(x)

Renvoie le nombre d'éléments ayant la valeur x dans la liste.

Listing 9: list.count(x)

```
1 >>> liste = ["a", "a", "a", "b", "c", "c"]
2 >>> liste.count("a")
3 3
4 >>> liste.count("c")
5 2
6
```

list.sort(*, key=None, reverse=False)

Ordonne les éléments dans la liste.

Listing 10: Ordonne les éléments dans la liste

```
1 # Création d'une liste de nombres premiers
2 prime_numbers = [11, 3, 7, 5, 2]
3
4 # Tri de la liste en ordre croissant
5 prime_numbers.sort()
6 print(prime_numbers) # Output: [2, 3, 5, 7, 11]
7
8 # Fonction de tri basée sur le second élément
9 def takeSecond(elem):
10     return elem[1]
11
12 random = [(2, 2), (3, 4), (4, 1), (1, 3)]
13 random.sort(key=takeSecond)
14 print('Sorted list:', random)
15
```

list.reverse()

Inverse l'ordre des éléments dans la liste.

Listing 11: Inverse l'ordre des éléments dans la liste

```
1 # Création d'une liste de nombres premiers
2 prime_numbers = [2, 3, 5, 7]
3
4 # Inversion de l'ordre des éléments
5 prime_numbers.reverse()
6 print('Reversed List:', prime_numbers) # Output: [7, 5, 3, 2]
7
```

list.copy()

Renvoie une copie superficielle de la liste. Équivalent à `a[:]`.

Listing 12: list.copy()

```
1 # Création d'une liste mixte
2 prime_numbers = [2, 3, 5]
3
4 # Copie de la liste
5 numbers = prime_numbers.copy()
6 print('Copied List:', numbers) # Output: [2, 3, 5]
7
```


Chaînes de caractères en Python

- ▶ Une chaîne de caractères est une séquence immuable de caractères.
- ▶ Elle peut être définie à l'aide de guillemets simples ou doubles.
- ▶ Les opérations sur les chaînes incluent la concaténation, l'extraction de sous-chaînes et les transformations (majuscule, minuscule, remplacement, etc.).

Concaténation et extraction de sous-chaînes

Listing 13: Concaténation et slicing

```
1 # Définition d'une chaîne
2 s = "Hello"
3
4 # Concaténation
5 s2 = s + " World"
6 print(s2) # Affiche "Hello World"
7
8 # Extraction d'une sous-chaîne (slicing)
9 print(s2[0:5]) # Affiche "Hello"
10 print(s2[-5:]) # Affiche "World"
11
```

Méthodes utiles sur les chaînes de caractères

Listing 14: Quelques méthodes utiles

```
1 # Convertir en majuscule et minuscule
2 s = "Python"
3 print(s.upper()) # "PYTHON"
4 print(s.lower()) # "python"
5
6 # Remplacer une partie de la chaîne
7 phrase = "Bonjour tout le monde"
8 nouvelle_phrase = phrase.replace("Bonjour", "Salut")
9 print(nouvelle_phrase) # "Salut tout le monde"
10
11 # Vérifier la présence d'un mot dans une chaîne
12 print("Python" in "J'aime Python") # True
13
```

Échapper des caractères spéciaux

Listing 15: Utilisation des caractères spéciaux

```
1 # Utilisation du caractère d'échappement \ pour afficher des
   guillemets
2 s = "Il a dit: \"Bonjour\""
3 print(s) # Affiche: Il a dit: "Bonjour"
4
5 # Nouvelle ligne et tabulation
6 s2 = "Ligne 1\nLigne 2"
7 print(s2)
8
```

Séparation et jointure des chaînes

Listing 16: split() et join()

```
1 # Séparation d'une chaîne
2 phrase = "Python est génial"
3 mots = phrase.split() # ['Python', 'est', 'génial']
4
5 # Jointure d'une liste de mots
6 nouvelle_phrase = "- ".join(mots)
7 print(nouvelle_phrase) # "Python - est - génial"
8
```

Utilisation des f-strings

Listing 17: Formatage avec f-strings

```
1 nom = "Alice"
2 age = 25
3
4 # Affichage avec formatage dynamique
5 message = f"{nom} a {age} ans."
6 print(message)  # "Alice a 25 ans."
7
```

Itération et vérifications sur les chaînes

Listing 18: Boucles et vérifications

```
1 # Parcourir une chaîne
2 s = "Python"
3 for lettre in s:
4     print(lettre)
5
6 # Vérifications
7 print("123".isdigit()) # True
8 print("Hello".isalpha()) # True
9 print("   ".isspace()) # True
10
```

Tuples et Ensembles

- ▶ **Tuples** : Séquences immuables, utiles pour stocker des données qui ne doivent pas être modifiées.
- ▶ **Ensembles** : Collections non ordonnées d'éléments uniques, idéales pour tester l'appartenance et effectuer des opérations ensemblistes.

Création et utilisation des tuples

Listing 19: Définition d'un tuple

```
1 # Définition d'un tuple
2 mon_tuple = (1, "ok", "olivier")
3 print(mon_tuple) # (1, 'ok', 'olivier')
4
5 # Tuple avec une seule valeur
6 mon_tuple = ("ok",)
7 print(type(mon_tuple)) # <class 'tuple'>
8
```

Affectation multiple et retour de fonction

Listing 20: Affectation multiple avec un tuple

```
1 # Affectation multiple
2 v1, v2 = 11, 22
3 print(v1) # 11
4 print(v2) # 22
5
6 # Retourner plusieurs valeurs avec une fonction
7 def donne_moi_ton_nom():
8     return "olivier", "engel"
9
10 print(donne_moi_ton_nom()) # ('olivier', 'engel')
```

Opérations sur les ensembles

Listing 21: Manipulation d'un ensemble

```
1 # Création d'un ensemble
2 nombres = {1, 2, 3, 3, 2}
3 print(nombres) # {1, 2, 3}
4
5 # Opérations sur ensembles
6 a = {1, 2, 3}
7 b = {2, 3, 4}
8 print("Union :", a.union(b)) # {1, 2, 3, 4}
9 print("Intersection :", a.intersection(b)) # {2, 3}
10
```

Ajout et suppression dans un ensemble

Listing 22: Ajout et suppression d'éléments

```
1 # Ajouter un élément
2 mon_ensemble = {1, 2, 3}
3 mon_ensemble.add(4)
4 print(mon_ensemble)  # {1, 2, 3, 4}
5
6 # Supprimer un élément
7 mon_ensemble.remove(2)
8 print(mon_ensemble)  # {1, 3, 4}
9
```

Différences et comparaisons d'ensembles

Listing 23: Différences et comparaisons

```
1 # Différence entre ensembles
2 a = {1, 2, 3, 4}
3 b = {3, 4, 5, 6}
4 print("Différence a - b:", a - b) # {1, 2}
5 print("Différence b - a:", b - a) # {5, 6}
6
7 # Sous-ensemble et sur-ensemble
8 print(a.issubset({1, 2, 3, 4, 5})) # True
9 print(a.issuperset({2, 3})) # True
10
```

Les dictionnaires en Python

- ▶ Un dictionnaire stocke des paires clé-valeur.
- ▶ Les clés sont uniques et permettent d'accéder rapidement aux valeurs associées.
- ▶ On peut ajouter, modifier, supprimer des paires et itérer sur ses éléments.

Créer et manipuler un dictionnaire

Un dictionnaire permet de stocker des données sous forme de paires clé-valeur, facilitant ainsi leur récupération rapide.

Listing 24: Définition et accès aux valeurs

```
1 # Création d'un dictionnaire
2 etudiant = {'nom': 'Alice', 'age': 22, 'filiere': 'Informatique'}
3
4 # Accès à une valeur
5 print(etudiant['nom']) # Alice
6
7 # Ajout d'une paire clé-valeur
8 etudiant['ville'] = 'Paris'
9
10 # Suppression d'une paire
11 del etudiant['age']
12
13 # Itération sur le dictionnaire
14 for cle, valeur in etudiant.items():
15     print(cle, ":", valeur)
16
```

Initialisation et mise à jour d'un dictionnaire

Un dictionnaire peut être initialisé vide et mis à jour dynamiquement en ajoutant des paires clé-valeur.

Listing 25: Créer et mettre à jour un dictionnaire

```
1 # Création d'un dictionnaire vide
2 mon_dict = {}
3
4 # Ajout de valeurs
5 mon_dict["nom"] = "Wayne"
6 mon_dict["prenom"] = "Bruce"
7 print(mon_dict)  # {'nom': 'Wayne', 'prenom': 'Bruce'}
8
```


Récupération sécurisée des valeurs

L'utilisation de 'get()' permet d'éviter les erreurs si une clé est absente en fournissant une valeur par défaut.

Listing 26: Utilisation de get()

```
1 # Utilisation de get() pour éviter les erreurs
2 personne = {"nom": "Wayne", "age": 45}
3 print(personne.get("nom")) # Wayne
4 print(personne.get("adresse", "Adresse inconnue")) # Adresse
   inconnue
5
```

Vérification de la présence d'une clé

On peut vérifier si une clé existe dans un dictionnaire avec l'opérateur 'in', ce qui est utile avant d'accéder à une valeur.

Listing 27: Vérifier si une clé existe

```
1 # Vérifier si une clé est présente
2 mon_dict = {"nom": "Wayne", "age": 45}
3 print("nom" in mon_dict) # True
4 print("adresse" in mon_dict) # False
5
```

Suppression et parcours d'un dictionnaire

On peut supprimer une entrée d'un dictionnaire à l'aide de 'del', puis parcourir l'ensemble des clés et valeurs avec 'items()'.

Listing 28: Supprimer et parcourir un dictionnaire

```
1 # Suppression d'une clé
2 mon_dict = {"nom": "Wayne", "age": 45}
3 del mon_dict["age"]
4 print(mon_dict)  # {'nom': 'Wayne'}
5
6 # Parcours des clés et valeurs
7 for cle, valeur in mon_dict.items():
8     print(f"{cle} : {valeur}")
9
```

Utilisation des méthodes `keys()` et `values()`

La méthode '`keys()`' permet de récupérer uniquement les clés du dictionnaire, tandis que '`values()`' permet de récupérer uniquement les valeurs.

Listing 29: Récupération des clés et valeurs

```
1 # Récupérer les clés
2 donnees = {"nom": "Wayne", "prenom": "Bruce"}
3 for cle in donnees.keys():
4     print(cle) # Affiche "nom", "prenom"
5
6 # Récupérer les valeurs
7 for valeur in donnees.values():
8     print(valeur) # Affiche "Wayne", "Bruce"
9
```

Utilisation de items() et combinaison Tuple/Dictionnaire

La méthode 'items()' permet de récupérer à la fois les clés et les valeurs sous forme de tuple. On peut aussi utiliser un tuple comme clé dans un dictionnaire.

Listing 30: Parcourir un dictionnaire avec items()

```
1 # Parcourir un dictionnaire avec items()
2 donnees = {"nom": "Wayne", "prenom": "Bruce"}
3 for cle, valeur in donnees.items():
4     print(cle, valeur) # Affiche "nom Wayne", "prenom Bruce"
5
6 # Utiliser un tuple comme clé
7 dico_coords = {}
8 dico_coords[(3,2)] = 12
9 dico_coords[(4,5)] = 13
10 print(dico_coords) # {(3,2): 12, (4,5): 13}
11
```

Copie et fusion de dictionnaires

Lorsqu'on affecte un dictionnaire à une autre variable, les modifications affectent les deux. Pour éviter cela, on utilise `'copy()'`. La méthode `'update()'` permet de fusionner deux dictionnaires.

Copie et fusion de dictionnaires

Listing 31: Copier et fusionner des dictionnaires

```
1 # Mauvaise copie (liée à la même référence mémoire)
2 d = {"k1": "Bruce", "k2": "Wayne"}
3 e = d
4 d["k1"] = "BATMAN"
5 print(e) # {'k1': 'BATMAN', 'k2': 'Wayne'}
6
7 # Bonne copie (indépendante)
8 d = {"k1": "Bruce", "k2": "Wayne"}
9 e = d.copy()
10 d["k1"] = "BATMAN"
11 print(e) # {'k1': 'Bruce', 'k2': 'Wayne'}
12
13 # Fusionner deux dictionnaires
14 a = {"nom": "Wayne"}
15 b = {"prenom": "Bruce"}
16 a.update(b)
17 print(a) # {'nom': 'Wayne', 'prenom': 'Bruce'}
18
```

Exercices pratiques

- ▶ **Exercice 1** : Créez une liste de nombres, ajoutez-y un élément, supprimez-en un autre et affichez la liste.
- ▶ **Exercice 2** : Manipulez une chaîne de caractères pour en extraire un mot à l'aide du slicing.
- ▶ **Exercice 3** : Créez un dictionnaire décrivant un étudiant (nom, âge, filière) et modifiez-le en ajoutant ou supprimant des informations.
- ▶ **Exercice 4** : Déclarez un tuple représentant des coordonnées et un ensemble contenant quelques valeurs, puis effectuez des opérations d'union et d'intersection sur des ensembles.

Conclusion du Chapitre 5

- ▶ Les listes permettent de manipuler des collections ordonnées et modifiables.
- ▶ Les chaînes de caractères offrent des opérations simples pour le traitement de texte.
- ▶ Les dictionnaires facilitent le stockage et l'accès aux données par paires clé-valeur.
- ▶ Les tuples et ensembles apportent des structures adaptées à des besoins spécifiques (immutabilité et unicité).

Pratiquez ces concepts avec des exercices pour renforcer votre compréhension !